OmniDoor3D User Guide

Table of Contents

Introduction 1

Using OmniDoor3D in URP or HDRP 2

Using OmniDoor3D Presets 2

OmniDoor3D Properties 3

Additional Information 8

OmniDoor3DConfig 11

Scripting Guide: OmniDoor3DController 11

Scripting Guide: Door Events 12

Adding Your Own Doors to OmniDoor3D 13

Introduction

Welcome to OmniDoor3D, your drop-in solution for a vast variety of working 3D doors - or windows, or almost any opening part of an environment or object! OmniDoor3D is not only a highly useful collection of existing doors - it is a modular system in which you can include your own doors with your own models, textures, and sounds.

OmniDoor3D includes many preset types of doors or windows, built with original code, 3D models and textures, which are yours to use. Sample SFX are also included using public domain or free license audio (see 'Docs\asset credits.txt' for info about sources and reuse). Paneled doors, windowed doors, garage doors, hatches, fence gates, portcullises, lids, and more are included as presets for you to use and learn from. And OmniDoor3D works properly at any position, scale, or rotation you need. You can also change the configuration of an OmniDoor3D at runtime and it dynamically updates - change the door type, sounds, behaviour, or visuals!

OmniDoor3Ds support locked and unlocked states. Doors can open / close automatically, and be controlled via script. Auto and controlled modes can operate concurrently. You can have a door controlled by multiple controllers. You can set any number of doors to the same controller. Controllers are easy to use, and examples for controllers are in the example scene. Additionally to the controller, several potentially useful door related events have been exposed for your scripts to respond to.

OmniDoor3D comes with some foundational motion types that are customizable in several ways. You can control the timing and speed of opening and closing. You can combine and nest doors and use motion types to create intricate and advanced doors, or even useful moving objects such as platforms or elevators. Doors can also be linked together so that they open and close at the same time, allowing even more complex door combinations such as folding, multi-sectioned, dilating, telescoping, rolling, and more. The limit is your creativity!

You can get started quickly by exploring the OmniDoor3D Examples scene (requires InputSystem package installed), or read on for more info.

Using OmniDoor3D in URP or HDRP

OmniDoor3D operates independently of rendering pipelines. However, materials included in the package are all very dependent on the rendering pipeline! By default they are in Built-In pipeline format. If you want to use OmniDoor3D and be able to load the example scene properly, you need to convert all the materials after installing the OmniDoor3D package. All materials are in the /Assets/Plugins/OmniDoor3D/Materials folder. Select them all, then select the Edit menu -> Rendering -> Materials -> “Convert Selected Built-In Materials to [URP/HDRP]”. Wait for the process to complete, and you’re all set.

Using OmniDoor3D Presets

Making use of OmniDoor3D’s included preset doors is a relatively simple matter of ensuring the available properties are set as you desire. Generally the process involves these steps:

- Add an OmniDoor3D prefab to your scene, set desired position, scale, and rotation. It has a default look. Don’t worry - the OmniDoor3D dynamically changes the door based on your settings. Also, you can safely override the prefab defaults, it won’t break it.
- Set remaining options: Door, Opening, Closing, Motion, and Sound.

The options are detailed in the next section. You can also follow along with a video playlist on [YouTube](https://www.youtube.com/playlist?list=PLNkmJbY8gMh7bwDooMwZoIwuFYfCHbF0_) (https://www.youtube.com/playlist?list=PLNkmJbY8gMh7bwDooMwZoIwuFYfCHbF0_).

OmniDoor3D Properties

Door Options	
Door Preset	Select from this drop-down list to choose a preset door, which are fully working doors with customization options. The type of door may affect the availability of other choices in this section, e.g. post related options are only shown for fence gates.
Door Materials	Upon selecting a door preset this will show a list of the default materials to be used. You can choose other materials to alter the look of the door.
Controlling Objects	Controlling objects are optional. If specified, this door will subscribe itself to the events of the chosen game objects having an OmniDoor3DController component. The events are OpenDoor, CloseDoor, ToggleDoor, LockDoor, and UnlockDoor. Your scripts can invoke these events from an OmniDoor3DController to perform the indicated action on all doors controlled by that controller.
Linked Doors	Add other OmniDoor3Ds to this list to have them open, close, lock, and unlock at the same time as this door.
Door Layer	This is the layer to use for door, handle, and fence post meshes.
Handle Type	Select from this drop-down list to choose the handles, if any, that you want for the door.
Handle Materials	If handles are chosen, this list of default materials will be shown. You can choose other materials to alter the look of the handle.
Post Layout	If the selected preset door is set up as a fence gate, you can choose which fence posts to add, if any.
Post Materials	These are the default materials for the fence posts. You can choose other materials to alter the look of the posts.
Windowed	The ‘Standard’ door presets have an optional window. Toggle it using this checkbox.

Opening Options	
Seconds to Open	The number of seconds it will take for the door to go from fully closed to fully opened.
Open Delay (s)	If a value is set here, when the door successfully opens, it is delayed by this number of seconds. The DoorWillOpen event doesn't fire until after this delay.
Triggered Open	If checked, the door will automatically open when a collider in the Triggering Layers enters the trigger. Note that a triggered open door does not automatically close with a lack of trigger presence, unless you set that option as well.
Trigger Shape	The desired shape of the trigger - Sphere or Box. The same Trigger Shape is used for opening and closing.
Trigger Radius	If you choose a Sphere trigger, you can set the radius of the sphere. The center of the sphere will be the center of the door mesh's bounds. The same Trigger Radius is used for opening and closing.
Trigger Extents	If you choose a Box trigger, you can set its extents. This is how far the box extends from the center of the door mesh's bounds, in the given axes. (N.B.: size = 2 * extents). The same Trigger Extents are used for opening and closing.
Trigger Offset	Set this Vector3 to offset the center of the trigger from the center of the door mesh's bounds. The same Trigger Offset is used for opening and closing.
Triggering Layers	You can specify which layers will trigger the door. The same Triggering Layers are used for opening and closing.
Start Locked	Locking is only supported in script. You can have the door be in the locked state when Play mode is entered, by checking this checkbox. To lock or unlock an OmniDoor3D at runtime, you must use an OmniDoor3DController.
Closing Options	
Seconds to Close	The number of seconds it will take for the door to go from fully opened to fully closed.

Close Delay (s)	If a value is set here, when the door successfully closes, it is delayed by this number of seconds. The DoorWillClose event doesn't fire until after this delay.
Triggered Close	If checked, the door will automatically close when it is open and there are no colliders in the Triggering Layers present within the trigger.
Trigger Shape	The desired shape of the trigger - Sphere or Box. The same Trigger Shape is used for opening and closing.
Trigger Radius	If you choose a Sphere trigger, you can set the radius of the sphere. The center of the sphere will be the center of the door mesh's bounds. The same Trigger Radius is used for opening and closing.
Trigger Extents	If you choose a Box trigger, you can set its extents. This is how far the box extends from the center of the door mesh's bounds, in the given axes. (N.B.: size = 2 * extents). The same Trigger Extents are used for opening and closing.
Trigger Offset	Set this Vector3 to offset the center of the trigger from the center of the door mesh's bounds. The same Trigger Offset is used for opening and closing.
Triggering Layers	You can specify which layers will trigger the door. The same Triggering Layers are used for opening and closing.
Timed Close	The door can be set to close after a number of seconds. This is independent of controlled or triggered closing.
Close After (s)	Number of seconds to wait after the door becomes fully open, upon which the door will begin closing.
Motion Options	
Motion Type	Choose from this drop-down list to pick a style of motion for opening and closing the door. Further options shown in this section will depend on the Motion Type.

Swing Direction	Only shown for Motion Type: Swing. Auto: The door will swing away from the triggering object. Forward: The door will swing toward positive local axes. Backward: The door will swing toward negative local axes.
Reverse Swing	Only shown for Motion Type: Swing. If checked, the door will swing in the opposite direction calculated. This is useful for quick fixes due to unusual rotation or without having to modify a mesh.
Swing Amount X / Y / Z	Only shown for Motion Type: Swing. This is the amount out of one full rotation that the door will rotate in the given axis, in the open state. For example, a typical swinging door would have 0 X and 0.25-0.45 Y and 0 Z rotation.
Slide Direction	Only shown for Motion Type: Slide. This is a Vector3 that specifies the direction the door will move when opening.
Slide Distance	Only shown for Motion Type: Slide. This is the number of local units that the door will move in the Slide Direction when opening.
Offset Slide	Only shown for Motion Type: Slide. If this checkbox is checked, you can have the door move to an offset from its position before sliding.
Offset Direction	Only shown for Motion Type: Slide. If using an offset slide, set this Vector3 to specify the direction of the offset.
Offset Distance	Only shown for Motion Type: Slide. If using an offset slide, set this value to specify the number of local units that the door will move in the Offset Direction before sliding.
Offset Portion	Only shown for Motion Type: Slide. If using an offset slide, set this value to specify how much of the opening or closing time is taken by the offset part of the motion. For example, if your Seconds to Open is 2, and Offset Portion is 0.2, the offset motion will be done in 0.4s and the main part of the slide in 1.6s.

Sound Options	
Play Audio	Check this checkbox to automatically play audio for the door events. Default sounds are included - you do not need to set custom sounds. For even more control over the sound, disable this checkbox and use the door events as a starting point. Note that audio plays at the beginning of an event, that is, the opening sound will be played when the door starts to open, not after it's fully open.
Custom Close Sound	Specific audio clip to play when closing.
Custom Open Sound	Specific audio clip to play when opening.
Custom Locked Sound	Specific audio clip to play when the door is attempting to open, but is locked.
Custom Locking Sound	Specific audio clip to play when the door is being locked.
Custom Unlocking Sound	Specific audio clip to play when the door is being unlocked.

Additional Information

- OmniDoor3D is compatible with all versions of Unity from 2021.2.0 forward.
- OmniDoor3D is compatible with and operates the same in any Unity rendering pipeline - Built-In, Universal, or High Definition. Example scenes and materials are provided for all three. The URP and HDRP scenes and materials should be manually extracted by double-clicking on AdditionalRenderingPipelineAssets.unitypackage.
- The OmniDoor3D prefab has a certain structure that is required for proper functioning. Should your prefab be inadvertently deleted or corrupted, you can rebuild it using the following structure.
  -

Create GameObject "OmniDoor3D" (Static) - You should reset the transform so the position and rotation are 0, and the scale is 1.	
Add Component: OmniDoor3D.cs script	No config required
Add Component: AudioSource	Set mixer group if desired

    -

Create GameObject "FirstPivot" (Static) - Child of "OmniDoor3D" - Reset the transform

      -

Create GameObject "InternalPivot" - Child of "FirstPivot" - Reset the transform

        -

Create GameObject "Door" - Child of "InternalPivot" - Reset the transform - The components you add will be given valid configurations the first time you select a door type	
Add Component: MeshFilter	No config required
Add Component: MeshRenderer	No config required
Add Component: MeshCollider	No config required

- After replicating the above structure, there is no door until you choose any door type from the Door Preset drop-down. Then you can save “OmniDoor3D” as a prefab. It doesn’t matter what initial settings you choose now for the prefab - upon instantiating the prefab, all options will work when selected.
- As noted in the prefab structure, you can set the Audio Mixer group of the AudioSource on the OmniDoor3D GameObject to have the sound controlled by your sound setup.
- The AudioSource is set to a fully 3D Spatial Blend with a fairly close falloff distance. The farther away your scene’s AudioListener is from a door, the lower its sound volume will be (to a minimum). You can put the AudioListener on the player for “realistic” dynamic volume levels. You can also set the AudioSource’s Spatial Blend to 2D if you want it to always play at full volume.
- Layer Notes:
    - The OmniDoor3D prefab’s root GameObject is in the IgnoreRaycast layer so that its collider does not interfere with your physics at all. That collider is only used for triggered opening and closing.
    - All hierarchy children of an OmniDoor3D’s root GameObject are dynamically placed in the layer specified by the Door Layer setting.
- For doors or objects that combine multiple OmniDoor3Ds:
    - Consider whether they all need to play audio, or only some, or just one.
    - Doors other than the root door may need to have their Triggered Open and Triggered Close turned off, and their Controlling Objects cleared, then linked to the root door, so they only open and close when the root door does.
    - Locked or unlocked state still matters for linked doors - if a door is locked, it won’t open even if it is linked to a door that opens.
    - When nesting doors, place the nested door in the InternalPivot of FirstPivot, so it is a sibling of Door.
    - When nesting doors, it’s best to have the trigger collider only on the root door and link the others (use timing options to control sequences). This is because the root door does not move, but the nested ones may, so the trigger collider stays in place if on the root door only. You can use the Trigger Offset and Extents / Radius to
- When setting up doors, and testing them in Play mode, if you get strange occurrences like a door rapidly opening and closing, it is a good idea not to have a door selected in the hierarchy while it moves - the reason being, when selected, due to the OmniDoor3DInspector script, the trigger collider will dynamically update its

center to the center of the door - you want it to stay in the original position! You can make runtime changes, just unselect the door when testing the motion.

- An OmniDoor3D's objects are all static except for InternalPivot. You can un-mark them static if you need those specific GameObjects to move.
- Invisible doors can be used as utility objects to take advantage of their motion ability, and have other doors nested inside, to create door types that are not possible with just one door. Simply disable the mesh renderer and mesh collider component of Door. Check the Rolling Door example in the OmniDoor3D Examples scene for how this might be useful.
- OmniDoor3D does not use Unity's Animation system to move objects - it directly manipulates their transforms.

OmniDoor3DConfig

The OmniDoor3D system has a few settings. They are in the OmniDoor3DConfig scriptable object in the Resources folder.

- **Check Time:** This is how long to wait between Update polls for doors responding to a need to close. Default value = 0.05 (20 checks per second). You can change it, and it will affect the response time between needing to close and starting the close process (which may have a delay of its own). I recommend not to change it if you want normal, smooth motion.
- **DB List:** This is a list of door databases in your installation. There should always be at least 2, OmniDoorDB and AddedDoorDB. You should not change this list.

Scripting Guide: OmniDoor3DController

The way to control an OmniDoor3D through script is via the OmniDoor3DController. OmniDoor3DController is simply a group of delegates that your scripts can invoke to control the door. Add an OmniDoor3DController script component to any GameObject and access it in your scripts. You then should add the OmniDoor3DController's GameObject to the Controlling Objects list for each door that you want to respond to these controls. The following delegates can be invoked:

- OpenDoor: All doors controlled by this controller will open unless locked.
- CloseDoor: All doors controlled by this controller will close.
- ToggleDoor: All doors controlled by this controller will toggle their state. I.e., if a door is closed and not locked, it will open. If a door is open, it will close.
- LockDoor: All doors controlled by this controller will be set to locked. If you lock an open door, it stays open, and can be closed, after which it cannot be opened again until unlocked.
- UnlockDoor: All doors controlled by this controller will be unlocked.

Example - Open from a script

```
using ABCodeworld.OmniDoor3D;
…
// Simplified for brevity - plz use a cached reference
if (openLogicSatisfied)
    GetComponent<OmniDoor3DController>().OpenDoor?.Invoke();
…
```

Scripting Guide: Door Events

Any OmniDoor3D will generate the following events that your scripts can listen to, so you can customize game behaviour regarding door stuff happening.

- DoorOpened(OmniDoor3D door) - fires any time the door goes from not fully opened, to fully opened.
- DoorWillOpen(OmniDoor3D door) - fires any time the door is about to start moving toward open. That includes from a fully closed state, and reversing during closing.
- DoorOpenFailed(OmniDoor3D door) - fires any time the door was attempted to be opened, but couldn't because it was locked.
- DoorClosed(OmniDoor3D door) - fires any time the door goes from not fully closed, to fully closed.
- DoorWillClose(OmniDoor3D door) - fires any time the door is about to start moving toward closed. That includes from a fully open state, and reversing during opening.
- DoorLocked(OmniDoor3D door) - fires any time the door goes from unlocked to locked.
- DoorUnlocked(OmniDoor3D door) - fires any time the door goes from locked to unlocked.

Example - Respond to door event

```
using ABCodeworld.OmniDoor3D;
…

void Start()
{
    OmniDoor3D cellDoor = GameObject.Find("JailCellDoor003");
    cellDoor.DoorOpenFailed += OnPrisonerMisbehave;
}

void OnPrisonerMisbehave(OmniDoor3D door)
{
    SummonGuard(door.transform.position)
}

void OnDestroy()
{
    cellDoor.DoorOpenFailed -= OnPrisonerMisbehave;
}
…
```

Adding Your Own Doors to OmniDoor3D

With not unreasonable effort, you can set up your own original doors and make them appear in the Door Presets list. Here is a step-by-step guide on how to do so. There is also a video guide located on YouTube (https://youtu.be/lPAYkgwPPsQ).

1. Pick your door model - this can be any mesh you want.
2. Prepare any materials, textures, sounds, etc. that you are going to use.
3. Once ready, edit the OmniDoor3DData.cs script. Near the beginning of the namespace, you will find the DoorKind enum. You should add a new element to the enum to represent your new door type, following the format shown in the script. Do not change the order of the enum, or you will break existing door presets. The Door Presets list is sorted alphabetically by the OmniDoor3DInspector script, so you only need to insert your new element at the end (the trailing comma is ok, and makes it convenient when adding a new type).
4. Save the OmniDoor3DData.cs script and let the Unity editor recompile scripts.
5. In the Added Door database, a ScriptableObject located at OmniDoor3D/Resources/AddedDoorDB, add entries for your new door. Do not modify the main OmniDoorDB, as it may be overwritten in updates, and you would lose any changes you made. In AddedDoorDB:
    a. Add a new element to “Door List” using the + button at the end of the list. Door List settings:
        i. **Door Kind** - Set this to the new DoorKind you just added in the enum.
        ii. **Door Mesh** - Set this to the mesh asset you want to use. OmniDoor3D will automatically use this mesh in the components of the door.
        iii. **Door Materials** - This should be a list of materials needed by your mesh. Note that if you are setting up a new preset based on the included Door mesh (used for Standard doors), only specify 4 of the 5 materials, because OmniDoor3D knows to use the 1st material as a 5th material only if the door has no window. If you are using any other mesh, you should specify one material per submesh. These will be the default materials whenever you make a door of this type, and can be overridden on any prefab instance.
        iv. **Door Sounds** - You can specify the default sounds for this kind of door, which can be overridden on any prefab instance.

v. **Local Position, Rotation, and Scale** - OmniDoor3D will place your mesh at the specified local position, rotation, and scale within the OmniDoor3D's internal pivot GameObject, so the position of your mesh's pivot with its bounds and its natural orientation and dimensions are of the utmost importance. These settings allow you to have your mesh positioned by OmniDoor3D exactly as needed so that when the OmniDoor3D moves the internal pivot, the door moves the way you want.

vi. **Handles** - Since doors could come in almost any shape or size, no handle could be positioned the same way or even make sense to be present on every existing type of door. Thus, you should add the handle kinds you want to support on your door into this list, and set the position, rotation, and scale appropriately (these values are in the door GameObject's child space).

b. If you are using the included Door mesh and want to support the window feature, you need to add your new DoorKind to the standardKinds array in OmniDoor3DData.cs at the beginning of the OmniDoor3DData class (this means they are just after the DoorKinds). Be sure to increase the declared size of the array.

c. If you are making a fence gate and want to support optional fence posts, you need to do two things:

   i. Add your new DoorKind to the fenceKinds array in OmniDoor3DData.cs at the beginning of the OmniDoor3DData class (this means they are right after the DoorKinds). Be sure to increase the declared size of the array.

   ii. You need to have a post prefab that has a Box Collider, and add a new element in the Post List in the AddedDoorDB resource. Make sure Door Kind and Post Prefab are pointing to your new assets. Set the positions of the left and right posts. These positions are within the same local space as the door itself.

d. To use an existing kind of handle, you do not need to add to the Handle List. But if you want to use a new kind of handle, you can. Update the HandleKind enum the same way as for DoorKind (see steps 3 - 4; the HandleKind enum is directly above DoorKind). Then create a prefab of your handle model. It doesn't need a collider. Connect the prefab to the new HandleKind by adding an element to the Handle List in the AddedDoorDB resource. Make sure Handle Kind and Handle Prefab are both pointing to the new values. For any

door you want to be able to use this handle, you need to add it to the Handles for that door in the main DoorInfo entry for that type of door.

   e. You must add your new door type to the motions you want it to support, by adding or updating one or more elements in the Motion List in the AddedDoorDB resource. Each motion type has a list of Allowed Door Kinds. You should add a new element to the Allowed Door Kinds list for each motion type you want to be available for your new door type. If the motion type you want isn't in the list, you can add a new element to the Motion List. Be sure to follow through by changing the value of the new element to your new door type.

6. All done! You can test your new door type by instantiating an OmniDoor3D prefab and selecting it from the Door Presets drop-down list.